

AI Arcade — Sprint Review Video Script (4:00)

[0:00–0:10] Open

AI Arcade for RetroMind Studios — Mastermind and Peg Solitaire, solved with classical AI. No neural nets, no guessing — math that's provably correct.

[0:10–1:00] Part 1 — The Detective Alibi (Mastermind)

[Show: heatmap animation]

Our Mastermind AI is demonstrating a live attempt to crack a hidden code. Each circle indicates a potential color-in-slot choice, with brightness reflecting the number of remaining hypotheses supporting it. As options are eliminated, they instantly turn dark.

Here's the secret: after each guess, the AI evaluates all 1,296 possible codes against one question — if this were the actual answer, would it generate the same feedback? If it wouldn't, then that candidate is mathematically impossible and is eliminated forever. The set of potential codes narrows each turn because we're not just guessing but systematically disproving options. This process prevents the AI from looping indefinitely.

[1:00–1:50] Part 2 — Rule-Based AI vs. LLMs

Why choose Mastermind over an LLM? Mastermind is fully constrained — each rule is precise, and every outcome is deterministic. Unlike a modern LLM, which provides approximations, our Constraint Satisfaction method guarantees a definitive proof of the answer.

An LLM functions as a statistical pattern-matcher. If you request it to monitor 1,296 hypotheses over six turns, it may hallucinate — misremembering eliminated codes or confidently suggesting guesses that contradict earlier feedback — because it predicts probable text rather than following a strict algorithm. For games with strict, precise rules, rigid logic is more effective.

[1:50–3:40] Part 3 — The Custom Heuristic (A*)

[Show: both bar charts]

Peg Solitaire presents a more challenging search problem. Blind BFS discovered a solution with 13 jumps but required exploring 3,012 states, keeping its entire frontier and visited set in memory—totaling over 5,700 states. DFS is less resource-intensive, examining only 490 states, but it relies on a fortunate branch without any certainty.

A* addresses this with an informed score, $f(n) = g(n) + h(n)$ — representing the cost so far plus the estimated remaining cost — and always expanding the most promising state first. My heuristic incorporates two penalties beyond the straightforward "pegs remaining" baseline: *isolated pegs*, which cannot jump if they have no neighbors, and *dispersion*, measuring how scattered the remaining pegs are, since the aim is to consolidate them into a single peg. This approach reduces the search space to 57 states — a 98% reduction compared to BFS.

One honest caveat: an *admissible* heuristic never overestimates true remaining cost, which guarantees the shortest solution. Mine isn't admissible — it deliberately overestimates. Normally that's risky. But here, every winning solution is forced to be exactly 13 jumps regardless of path, so there's no shorter solution to miss. I get to trade away that guarantee for free, and spend the savings entirely on speed.

[3:40–4:00] Close

In closing we can conclude that Constraint Satisfaction is used for pure logical deduction, while Informed Search is applied for pure combinatorial search — these are two traditional AI methods, and each is suited to its respective problem. Thank you!